

Selenium → Playwright Migration

Interview / review talking points · GHIX UI automation (Playwright + Cucumber) · September 2026

How to use this: Sections 1–3 are the core narrative — keep the first two minutes tight. Drop into later sections only if the panel probes. Replace the *[bracketed]* placeholders with your own numbers (weeks, % scenarios migrated, Jenkins job names) before you present.

1. The 30-second pitch

We did **not** rewrite the product tests from scratch. We **lifted the existing Cucumber/Gherkin + Page Object model from Selenium WebDriver to Playwright**, kept feature files and business steps as the contract, and replaced the browser layer underneath.

AI (Cursor) accelerated **mechanical conversion, locator cleanup, and Jenkins failure triage** — it did **not** decide what to assert. Humans still owned state tagging, config-driven UI behaviour, and the "fail vs skip" rules.

2. Why Playwright (expect this question first)

Business framing: faster CI, fewer flakes, one engine across browsers, and far better evidence when a state job fails at 2 a.m.

Area	Selenium (old)	Playwright (now)
Wait model	Implicit waits plus a lot of <code>WebDriverWait</code> and sleeps	Auto-wait plus explicit <code>waitFor*</code> in our <code>Browser.js</code>
Debugging	Screenshot and console log	Trace, video, Allure attachments
Parallel	Hard — one driver, shared session and cookies	Cucumber <code>--parallel</code> ; we run 16 workers (<code>SetUpData = 0</code> , <code>visual = 1</code>)
Iframes / tabs	Fragile	First-class page and context handling
Network	Limited visibility	Wait on load / spinner instead of guessing

What we deliberately kept: Gherkin, stepDef wording, the POM, `resources/selectors`, `configs.json`, Jenkins state tags such as `@OR_QEP_Regression`, and Allure reporting.

If asked "why not Cypress?" — Cypress is app-centric and weaker for multi-tab, multi-role GHIX journeys (consumer, CAP, assister, entity) and for a Java-style POM with Cucumber at this scale. Playwright mapped much closer to Selenium's driver model, which is exactly what made a layer swap possible.

3. How we migrated (strategy)

Approach: strangler / layer swap, not a big-bang rewrite.

- Inventory** — features, shared steps, Browser/Assert helpers, DB queries, Jenkins jobs per state.
- Keep the contract** — `.feature` files stay; product owners still read the same scenarios.
- Replace the driver** — `Browser.js` and `Assert.js` become Playwright-backed. Page models keep their method names, so step definitions barely change.

4. **Locators stay in JSON** — read via `browser.getSelector(...)`; no XPath inlined in JS; prefer `data-testid`.
5. **Pilot one module** (Plan Display / SSAP), then roll through the rest.
6. **Dual run** until the Playwright Jenkins jobs were green enough, then retire Selenium *per suite*, not overnight.
7. **Harden** — no hard waits, no try/catch in test code, no silent `log + return` inside a verify.

Definition of done: tagged regression runs on Jenkins for each exchange state, Allure publishing, coverage equal or better, and a measurably lower flake rate.

4. Architecture after migration

- **Cucumber** loads `Hooks.js` and `tests/stepDefs/**`.
- **Page models** under `tests/pagemodels/` — one place per screen.
- **Selectors** — `resources/selectors/common` plus `exchange/{STATE}` overrides.
- **Content** — expected text per language and state.
- **Data / config** — `configs.json` mirrors `gi_app_config`. Tests *read config, then assert the UI*; they never guess per state.
- **DB** — `PlanDBQueries` / `DbHelper` when we must prove data, not for every UI flag.
- **CI** — Jenkins per state (for example `OR_Regression_Release_Playwright_Testing`), Allure, `--retry 1`.

Good supporting story: an OR failure of "Compare Now button not visible" is a product or synchronisation issue surfaced by the suite — not evidence that "Playwright is broken".

5. Challenges and what we actually did

Waits and flakes

Selenium hid timing problems behind sleeps. Playwright is faster, so real races surfaced. We banned `waitForTimeout` / `sleep` in test code and centralised retries in `Browser.js` (`waitToBeClickableAndClick`, spinner-hide after every navigation or popup).

Implicit vs auto-wait

Engineers were clicking before the spinner cleared. The rule is now explicit: after every navigation or new page, call `waitForLoadingSpinnerToHideWithRetry()` before interacting.

try/catch hiding bugs

Legacy Selenium code caught timeouts and logged them, so Jenkins stayed green over real defects. We made this a hard rule: no try/catch in page models or steps. Optional UI uses `isElementDisplayed` and then asserts either present or deliberately absent.

Locators

Selenium-era `//*[@...]` XPath is slow and brittle. We moved to tag-specific XPath and `data-testid`. AI was strong at bulk-rewriting locators; humans reviewed uniqueness.

Async / await

Moving from synchronous Java-style Selenium to JS promises. The most common human error was a missing `await`. Lint rules plus "never use an empty `.catch()`" caught most of it.

Parallelism

Shared `global.updateDataJson`, the same user, the same DB row. `SetUpData` cannot run in parallel (`parallel: 0`); visual regression runs sequentially to avoid screenshot collisions; worker count is overridable when several states run at once.

Multi-state behaviour (GA, OR, NV, PA, ...)

Same scenario, different UI — NV, for example, has no doctor-search slide. The wrong fix is `if (NV) return;`. The right fix is to **untag NV** or **assert the field is not displayed**. Similarly, provider rows are config-driven (DOCTOR / FACILITY vs DENTIST) rather than hardcoded to three tiles.

Chakra / React (GA)

`waitForNetworkIdle` hung and the compare page bounced on reload. The state-specific wait is documented as a product workaround with a ticket reference — not an excuse for a sleep culture.

Skills

SDETs moving from Java to JS/Playwright. We paired on the first page-model conversion (with Cursor), then the engineer owned the next one unaided.

Reporting

Allure plus a custom `reporter.js`. Traces on failure are a genuine step up from Selenium screenshots.

6. AI: how we used it, and what we refused

Tooling: Cursor with repository rules in `.cursorrules`, plus chat-based Jenkins and Allure triage.

Where AI was genuinely fast (hours to minutes)

- Converting Selenium-shaped methods to Playwright `await browser.*` inside an existing POM.
- Adding selectors to the JSON files rather than inventing them inline in JS.
- Bulk Gherkin tagging (`@PlanDisplay_QEP_Tests_{state}`) across many feature files — mechanical, high file count.
- Drafting a DB query or a `configs.json` reader in the same style as its neighbours.
- Explaining a stack trace end to end: `Assert.assertTrue` → page-model method → the owning step.
- Git forensics, e.g. "why did this PR show 1 changed file when the commit had 36?" (three-dot vs two-dot diff, work already upstream).
- Epic analysis to find reusable existing steps instead of duplicating Gherkin.

Where AI was wrong — say this out loud, it builds credibility

- It **hardcoded** health vs dental provider rows when the product is **config-driven**. We reverted twice.
- It added `# State : GA, ID, ...` header comments, inflating the diff. We deleted them.
- It tried to skip NV with a log instead of untagging the scenario — a silent pass.
- It reached for a live DB query when the agreed source for that check was `configs.json` with a `common` key plus state overrides.

Guardrails (the "AI governance" answer)

- Cursor rules enforce: no sleeps, no try/catch, locators only in JSON, no `//*` XPath, no live `@HIX-####` tags, assertions rather than logs.
- Humans review *intent*: config vs DB, which states are tagged, and fail vs documented N/A.

- AI does not push code, does not bypass hooks, and never invents expected API error strings — those come from the feature file.

Stating the acceleration honestly

"Boilerplate conversion and bulk tagging were the long pole. AI cut that down so engineers spent their time on synchronisation, state exceptions, and assertion quality. The net effect was more scenarios migrated per sprint — not fewer people."

If they push for a number, quote only something you can defend, e.g. "the first page-object conversion went from about half a day to roughly one to two hours including review" — [confirm this is true for you].

7. Quality and flake: "is Playwright just more stable?"

Playwright is not magically stable. The stability came from how we used it:

- auto-wait combined with an enforced spinner contract
- retries in the browser layer, never scattered through scenarios
- `--retry 1` for genuine infrastructure blips
- treating missing UI as a failure whenever the tag says that state should have it
- config-driven expectations, so GA (two provider rows) and IL (three) do not share one rigid assertion

The flakes that remain are mostly product-side (Compare Now visibility, spinners, the Chakra bounce) — which is exactly the right split.

8. CI/CD and reporting

- Same Jenkins pattern as before: state + tag expression + Allure.
- Parallel 16, except SetUpData (0) and visual regression (1).
- Artifacts: Allure report, screenshots, traces. Emphasise that we *open the trace* rather than guess.
- Do not claim zero retries. `--retry 1` is the honest answer.

9. Coverage, coexistence and rollback

- Gherkin is the coverage ledger: if a scenario is tagged and green on Playwright, that case has migrated.
- Rollback plan: keep the Selenium job alive until the Playwright job is trusted for that module and state, then switch it off.
- Risk: two sources of truth. Mitigation: one feature file, one stepDef — only the engine changes.

10. Locators, accessibility, visual

- `data-testid` first; `axe-playwright` is already a dependency if accessibility comes up.
- Visual regression must run sequentially to avoid screenshot collisions — a useful detail showing you thought through parallel limits.

11. Team, process, "would you do it again?"

Would repeat: the layer swap while keeping Gherkin; a rules file to constrain AI; making the config/state matrix explicit.

Would avoid: letting AI invent skips; migrating locators into JS; running SetUpData in parallel; treating `configs.json` and the live DB as interchangeable without declaring the source of truth.

Source-of-truth answer (likely follow-up after the Plan Display example): `configs.json` is the automation mirror of `gi_app_config`. We verify the UI against that mirror so a tile check does not need a DB round trip. If the JSON drifts from the DB, that is a data-sync problem to raise openly — not something to paper over.

12. Rapid-fire questions and one-line answers

Question	Answer
Selenium vs Playwright?	Auto-wait, traces, real parallelism — while keeping Cucumber and the POM.
Rewrite or migrate?	Migrated the driver layer; kept the scenarios.
Biggest risk?	Silent passes, and state-specific UI treated as one uniform product.
Biggest win?	Parallel Jenkins runs, traces, and far less sleep-driven flake.
How did AI help?	Conversion, tagging, triage and first-draft page models — under explicit rules.
How did AI fail?	Wrong domain assumptions (hardcoded vs config), noisy diffs, skip-as-pass.
Hard waits?	Forbidden. Spinner contract plus <code>waitFor*</code> .
Why JS, not Java Playwright?	Aligns with this repo and the Node-based Jenkins pipeline. <i>[adjust if you also run Java]</i>
How do you know coverage is equivalent?	Same Gherkin and tags; compare Allure history against the old Selenium job.
Secrets and security?	No credentials committed in locators or config; environment variables and Jenkins credentials.
When is the migration finished?	When the Selenium job for that state/suite is retired and Playwright is the gate.

13. How to structure it in the room

1. **Problem** — Selenium was too slow and flaky for multi-state GHIX CI.
2. **Decision** — Playwright, while keeping Cucumber.
3. **Method** — wrapper first, then module by module.
4. **AI** — an accelerator governed by `.cursorrules`, not the owner of the design.
5. **Proof** — Playwright Jenkins jobs, Allure, and a concrete example (NV doctor page, config-driven provider rows).
6. **Lesson** — automation quality lives in assertions and tagging, not in the tool logo.